BrainAI

ProSuite Industrial

CLI & AUTOMATION

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**ProHash Command Line Reference**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2025-01-01	S. Köhne	Stable	Initial ProHash-256 Command Line Interface
2.0.0	2026-02-27	S. Köhne	Stable	Optimized I/O Streaming & Pipeline Support

1 Product Overview

The **ProHash Executable** (`prohash.exe` / `prohash`) is a minimalist, high-performance tool designed to generate cryptographic 256-bit digests. Based on the ProHash Sponge construct, it is specifically engineered for verifying large datasets in business-critical environments.

Core Features:

- **Complete Abstraction:** The CLI handles the entire management of the `ProHash_Ctx`. Users do not need to perform manual initialization or finalization steps.
- **Chunk-based Streaming:** Files are processed in optimized 16 KB blocks. This enables the hashing of multi-terabyte files with constantly low RAM usage (Zero-Footprint philosophy).
- **Pipeline & Redirect Support:** Supports both direct console output and writing to files for automated processing in script chains.
- **Zero-Trust Cleanup:** After calculation, the internal state of the engine is automatically overwritten with zeros (wiped) in RAM before the process terminates.

2. Technical Specifications

2.1 Global Constants

- **Hash Algorithm:** ProHash-256 (Custom Sponge "The Grinder").
- **Output Format:** 64-character hexadecimal string (corresponding to 32 binary bytes).
- **I/O Buffer:** 16,384 bytes (16 KB) per read cycle.

2.2 Command-Line Parameters (CLI)

The syntax is intentionally kept simple: `prohash <INPUT_FILE> [OUTPUT_FILE]`

- **INPUT_FILE**: Path to the file for which the hash is to be calculated.
- **OUTPUT_FILE** (Optional): Path to a file where the resulting hash string will be written. If this parameter is missing, the output is sent directly to `stdout`.

3. Global Error Management (Exit Codes)

Exit Code	*Meaning*	*Description*
0	SUCCESS	Hash successfully calculated and output.
1	ERR_PARAM	Missing or incorrect arguments during call.
2	ERR_IN_IO	Input file could not be opened or read.
3	ERR_OUT_IO	Output file could not be created or written to.

4. Linux / Bash Integration

Ideal for integrity checks after downloads or before backup processes.

4.1 Example: Integrity Check after Transfer

```bash
#!/bin/bash
FILE="database_dump.sql"
EXPECTED_HASH="7f83b165..."

# Calculate hash and store in variable
CURRENT_HASH=$(./prohash "$FILE")

if [ "$CURRENT_HASH" == "$EXPECTED_HASH" ]; then
    echo "Integrity confirmed: File is unchanged."
else
    echo "ALARM: Hash mismatch! File has been tampered with."
    exit 2
fi
```

5. Windows PowerShell Integration

5.1 Example: Batch Hashing of a Folder

```
$Files = Get-ChildItem "C:\Logs\*.log"
foreach ($File in $Files) {
    # Calculate hash and save to .hash file
    $HashPath = $File.FullName + ".hash"
    Start-Process -FilePath "prohash.exe" -ArgumentList "$($File.Full
    Write-Host "Hash created for $($File.Name)."
}
```

6. Best Practices & Security Guidelines

- **Automation:** Use the `OUTPUT_FILE` option when generating hashes for thousands of files in a loop to avoid `stdout` latencies.
- **Verification:** Always compare ProHash values via secured channels. An attacker who modifies the file could otherwise also manipulate the stored hash file.
- **Determinism:** ProHash-256 delivers identical values for the same input on any hardware (x86, ARM, RISC-V) and any operating system.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**